SkillTrend AI

What we built in Phases 1–3 — a plain-English recap

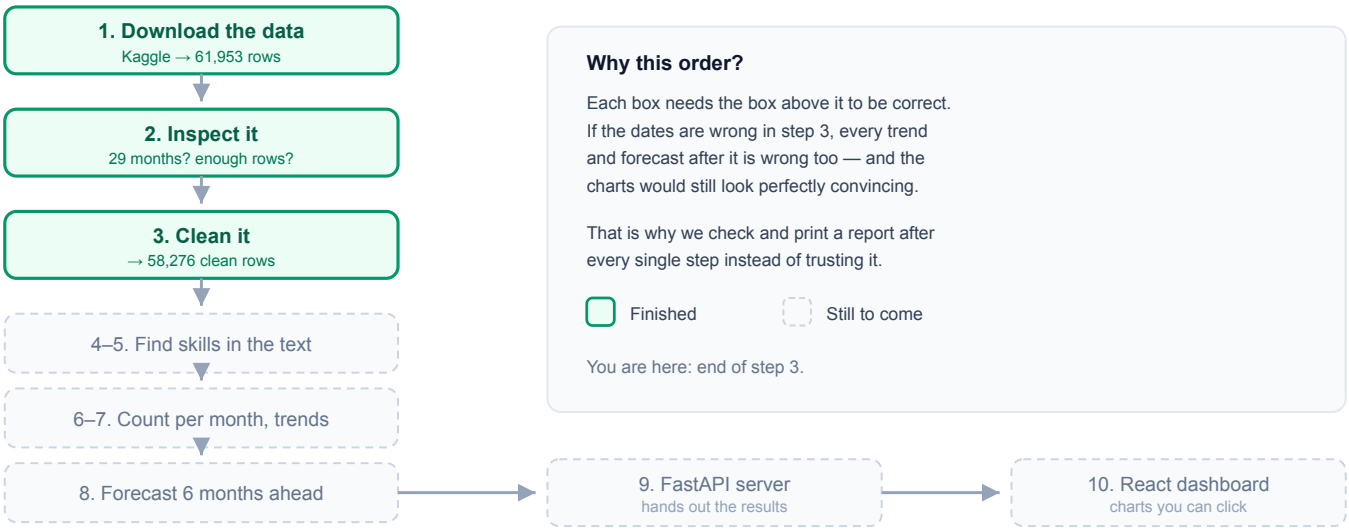
This project studies **real job advertisements** to answer four questions: which technical skills employers ask for most, how that changes month by month, which skills are rising or falling, and what the next six months might look like.

We are using **58,276 data-analyst job postings** collected between November 2022 and April 2025 — 29 months of history.

EVERYTHING HERE IS FREE

No paid services are used anywhere in this project. The dataset, all the software libraries, and the servers all run on your own laptop at zero cost.

The full pipeline (where we are now)



PHASE 1 · DONE

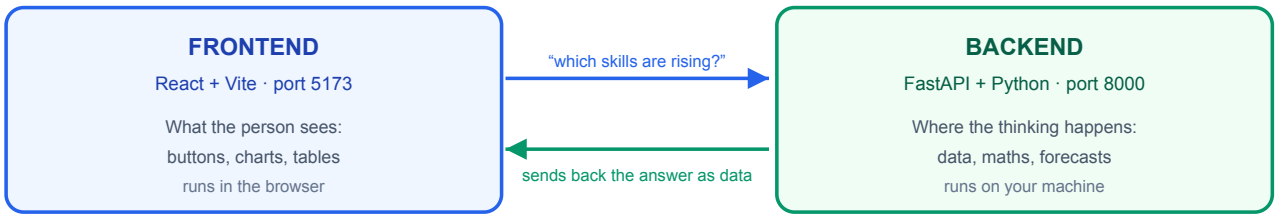
Setting up the workshop

Before writing any project code, we built the place the project lives in. Nothing clever happens here — but skipping it causes problems for months.

What we actually did

STEP	WHAT IT MEANS IN PLAIN WORDS
Renamed the folder	`SkillTrend AI` → `SkillTrend-AI`. The space in the name meant every terminal command needed quote marks around it.
Started Git	Git is a time machine for your code. It records every version, so a mistake is never permanent.
Made a `.gitignore`	A list of things Git should ignore — downloaded data, installed libraries, passwords. Without it you would upload 30,000 junk files to GitHub.
Created a virtual environment	A private, isolated copy of Python just for this project. Another project can use different library versions and neither breaks.
Built the backend	A small FastAPI server with one test address, `/api/health`, that replies “I am alive”.
Built the frontend	A React website that asks the backend how it is doing and shows the answer.

How the two halves talk



WHY SPLIT IT IN TWO?

The backend does slow, heavy work (reading 58,000 job ads, running maths). The frontend only draws pictures. Keeping them separate means the website stays fast, and you could later swap in a phone app without rewriting any of the analysis.

TWO ERRORS YOU HIT — BOTH NORMAL

Download kept timing out. Not a bug in your code — a slow internet connection. Fixed by telling the installer to wait longer.
“Address already in use”. An old server was still running and holding port 8000. Only one program can use a port at a time. Fixed by stopping the old one.

PHASE 2 · DONE

Getting the data — and testing whether it is good enough

We downloaded a real dataset from Kaggle, then did something most tutorials skip: **we checked whether the data can actually support the claims we want to make.**

The Data Gate

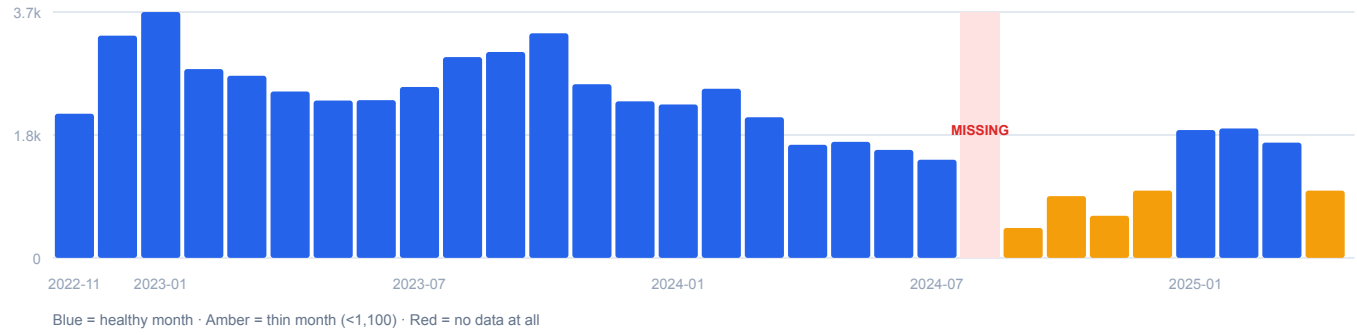
A forecast needs history. Predicting six months ahead from five months of data is not forecasting — it is guessing with a nice-looking chart. So before writing any prediction code, we set pass/fail rules *in advance* and measured the real file.

THE TEST	NEEDED	YOUR DATA	RESULT
Months of history	18 or more	**29**	PASS — real forecasting is honest
Job ads per month	300 or more	**2,298**	PASS — percentages are stable
Ads containing skills	70% or more	measured in Phase 5	still to check

DECIDING THE RULES BEFORE SEEING THE ANSWER

This is the important bit. If you look at the data first and then pick a threshold, you will unconsciously pick one the data passes. Writing the rule down first is what makes it a real test.

How many job ads we have each month



Blue = healthy month · Amber = thin month (<1,100) · Red = no data at all

TWO HONEST PROBLEMS IN THIS DATA

August 2024 is completely missing — the collector broke that month.
Autumn 2024 is thin — September has only 449 ads instead of the usual ~2,300, so its numbers are noisier.

We are not hiding either of these. Both go in the project documentation and will be shown on the dashboard, because a reader deserves to know which parts of a chart to trust.

The dataset we chose, and what we gave up

We picked a dataset of **data-analyst** job ads. It was a real trade-off:

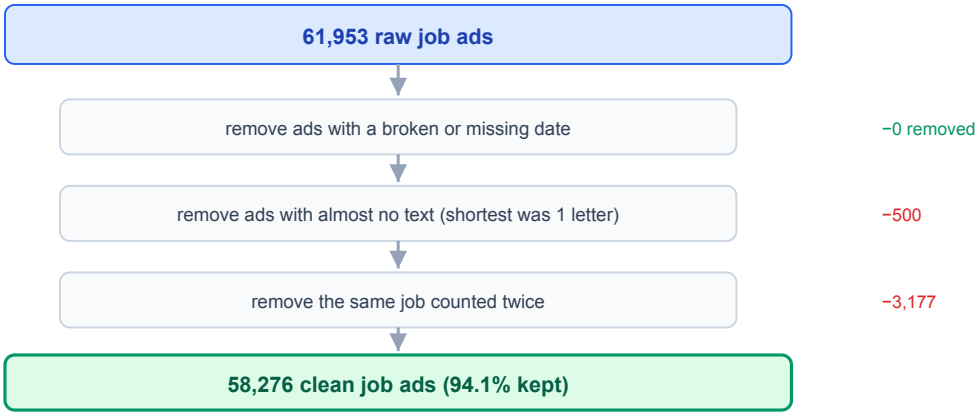
- **What we gained:** 29 months of continuous history — enough for genuine forecasting.
- **What we gave up:** web-developer skills like React and Kubernetes barely appear, because these are analyst jobs. So the project is honestly scoped as “skills in data-analyst roles”, not “all of tech”.

PHASE 3 · DONE

Cleaning the data

Raw data is messy. This step removed what we cannot use and put the rest into a tidy, consistent shape — and printed a receipt showing exactly what it removed.

What went in, and what came out



The five decisions inside the cleaning script

1. **Threw away 18 of the 27 columns.** Salary columns were 84–93% empty and travel time was 100% empty. A mostly-empty column cannot support any conclusion.
2. **Removed duplicate job ads.** The collector ran daily, so some jobs were recorded more than once. We kept the *earliest* sighting, which is closest to the day the job was really posted. Without this, popular long-running ads would be counted several times and distort every number.
3. **Made all text lowercase and removed web links.** So that “Python”, “python” and “PYTHON” count as one thing.
4. **Kept the punctuation — on purpose.** See the box below.
5. **Labelled each ad with its month**, ready for counting in Phase 6.

THE ONE PLACE WE IGNORED THE TEXTBOOK

Almost every text-cleaning tutorial says “remove all punctuation”. Here that would be a disaster:

```
C++ becomes c · .NET becomes net · Node.js becomes node js · CI/CD becomes ci cd
```

Those symbols *are* the skill name. Here punctuation is information, not noise. Knowing when standard advice does not apply to your problem is the real skill.

SOMETHING ELSE WE FOUND

We expected job descriptions to be full of HTML tags, so the plan included a tool for stripping them out. When we actually looked: **zero** ads contained HTML. So we deleted that tool from the project. Checking beats assuming.

Where we are, and what comes next

58,276	29	3	₹0
CLEAN JOB ADS	MONTHS OF HISTORY	PHASES DONE	TOTAL COST

Next up — Phase 4: teaching the computer to spot skills

Right now we have 58,276 paragraphs of English text. The computer does not know that “Power BI” is a skill and “the ideal candidate” is not. Phase 4 builds a **dictionary** of about 126 skills in 9 groups, and teaches the computer to find them.

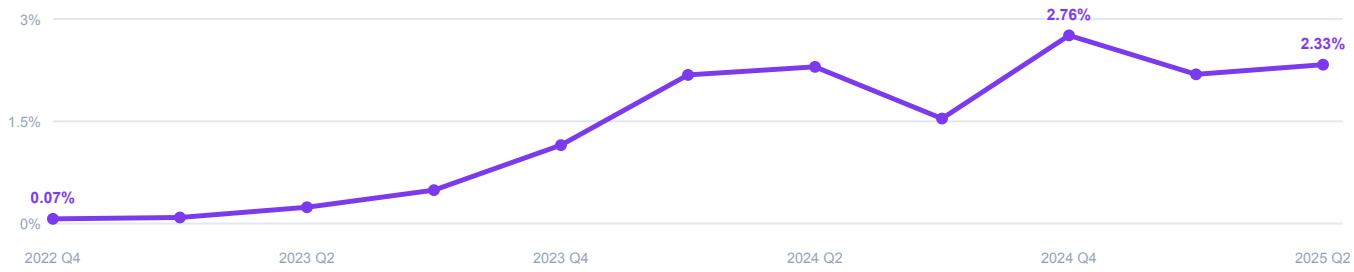
The hard part is words that mean two things:

THE TRAP	HOW WE HANDLE IT
“excel *at* teamwork”	That is the verb, not the Microsoft product. We ignore “excel” when the next word is “at” or “in”.
“*go* above and beyond”	Not the Go programming language. We only accept “golang” or “go programming”.
PowerBI / power-bi / Power BI	Three spellings, one skill. All three become “Power BI” so they are counted together.

A REAL DISCOVERY ALREADY WAITING IN YOUR DATA

The dataset came with its own list of skills — but that list was written in 2022 and contains **no AI terms at all**. No ChatGPT, no LLM, no LangChain. Your extractor will find what theirs structurally cannot. Here is what is hiding in the text:

Share of job ads mentioning generative AI



From almost nothing to roughly 1 ad in 40, in two years. This is measured from your own cleaned data — not an estimate.

Still to come

Phase 4–5	Build the skill dictionary and run it over all 58,276 ads
Phase 6–7	Count skills per month and work out what is rising and falling
Phase 8	Forecast the next 6 months — and check the model against a simple baseline
Phase 9–10	Build the API and the dashboard you can click through
Phase 11–12	Test everything end to end, then write the documentation

Words worth knowing

Every term we have used so far, in one place.

TERM	WHAT IT MEANS
Git	A time machine for code. Records every version so you can always go back.
GitHub	A website that stores a copy of your Git history online.
Repository (repo)	One project folder that Git is tracking.
Commit	One saved snapshot, with a note explaining what changed.
Virtual environment	A private copy of Python for one project, so projects cannot break each other.
Library / package	Code somebody else already wrote that you can use, like pandas.
pandas	The Python tool for working with tables of data. Like Excel, driven by code.
API	A menu of requests one program can make to another. Our dashboard uses it to ask the backend for numbers.
FastAPI	The Python tool we use to build that menu.
React	The tool we use to build the website you click on.
Port	A numbered door on your computer. The backend uses 8000, the website uses 5173.
CORS	A browser safety rule. Because our two halves use different ports, we had to explicitly allow them to talk.
CSV	A plain text table. Simple, but large and slow.
Parquet	A smarter table format. Smaller, much faster, and remembers whether a column holds dates or text.
Deduplicate	Remove repeated records so nothing is counted twice.
Pipeline	A chain of steps where each one feeds the next.
Forecast	An estimate of future values based on past ones. Always an estimate, never a fact.
Baseline	A deliberately simple prediction, used to check whether a clever model is actually any better.

THE PRINCIPLE BEHIND THIS WHOLE PROJECT

Keep three things clearly separated, and never blur them:

1. **What we observed** — counts taken straight from real job ads.
2. **What we calculated** — trends worked out from those counts.
3. **What the model predicted** — a guess about months that have not happened.

On the dashboard these will look different on purpose: solid lines for real history, dashed lines with a shaded band for forecasts. A project that says plainly what it does not know is worth more than one that looks impressive and cannot be defended.